FinalProject - Automation of Estimation Process

मोहन विश्वनाथ गई १६०.pdf
PDF

165 mohan vishwanath gai CHECK.xlsx
Spreadsheet

we have to automate the estimation calculation and like that kind of project one website or some applicaton which will make it easy to do that work and do automate the process this was the idea given by one person to us we are not more clear on this so the i will upload 2 documents regarding to it and read it throughly and tell me what what they expected in depth read hte excel file and tell us how can we proced do read more detiled all the pages in excel sheet and like that
Show moreShow less

Yes. I went through the uploaded PDF page-by-page and inspected the Excel workbook sheet-by-sheet, including the formulas and the relationships between the sheets.

The important point is this:

This is not simply a generic "estimation calculator."
The documents describe a fairly specific workflow for **valuation of an affected/encroached house under the Jigaon Project**, where the final payable estimate is derived from measurements, government schedule rates, depreciation, and a second valuation/salvage calculation.

The existing Excel file is essentially a partially automated version of that process. Your project opportunity is to turn this complicated Excel + handwritten/document workflow into a proper application.

---

1. What the documents actually represent

The PDF is a real government valuation file for:

- Jigaon Project
- Village: Dadulgaon
- Taluka: Jalgaon Jamod / Tq. Nandura context in the document
- District: Buldhana
- Owner: Mohan Vishwanath Gai
- House No.: 165
- L.A. Case: 15/2008-09

The first page identifies the work as valuation of houses/encroachment coming under the submergence of the Jigaon Project.
मोहन विश्वनाथ गई १६०
The PDF then contains several stages:

House/property information → construction measurements → item quantities → government rates → abstract estimate → depreciation → second/salvage valuation → final valuation → supporting photos/panchnama/drawing.

That is the core business process your application should automate.

2. What each PDF page is doing

I went through all 22 pages.

PDF page	What it contains	Importance for application
1	Cover/title, owner, house number, project, valuation purpose	Case creation
2	Formal estimate sheet, department/head information, owner/case information, applicable rate sources	Case metadata + rate policy
3	**Valuation by Depreciation**	Depreciation engine
4	**Valuation by Depreciation 2**	Second/salvage valuation + deduction
5	Abstract estimate, first group of construction items	Itemized estimate
6	Continuation of abstract + total	Itemized estimate
7	Abstract items 1–3	Construction estimate
8	Abstract items 4–9	Construction estimate
9	Abstract items 10–16	Construction estimate
10	Abstract item 17 + total	Construction estimate
11	Measurement Sheet items 1 and 4	Quantity calculation
12	Measurement calculations for more items	Quantity calculation
13	Measurement calculations	Quantity calculation
14	Measurement calculations	Quantity calculation
15	Measurement calculations	Quantity calculation
16	Measurement calculations	Quantity calculation
17	Final measurement item + signatures	Quantity calculation
18	House photograph	Evidence
19	Another house photograph	Evidence
20	Panchanama/property details	Supporting legal/document data
21	Continuation of Panchanama + signatures	Supporting data
22	Construction drawing/plan with dimensions	Source for measurements

So the application is not just calculating numbers. It must also maintain the **evidence and source information behind those numbers**.

3. The Excel file is the biggest clue

The Excel workbook contains these sheets:

1. `Cover Page`
2. `FC`
3. `Dep (2)`

4. **AB 10%**
5. **Dep**
6. **AB**
7. **mea**
8. **Final RA**
9. **Y.P.**
10. **Sheet1**
11. **Sheet2**

This reveals the original process very clearly.

The main dependency chain is roughly:

```
House / Measurement Input
          ↓
         mea
          ↓
         AB
          ↓
        Dep
          ↓
         FC
```

There is another path:

```
Selected salvage items
          ↓
       AB 10%
          ↓
       Dep (2)
          ↓
        FC
```

And the rate database is primarily:

```
Sheet1
   ↓
Rate lookup
   ↓
AB / AB 10%
```

Y.P. is a lookup table used by the depreciation calculation.

---

4. The most important sheet: `mea`

This is the **Measurement Sheet**.

This is where the actual physical dimensions of the building are converted into quantities.

For example, the Excel has calculations such as:

```
2 × 6.66 × 0.83 × 0.75 = 8.29 Cum
```

for long walls.

Then:

```
2 × 2.85 × 0.83 × 0.75 = 3.55 Cum
```

for short walls.

Then:

```
12 × 0.20 × 0.20 × 0.75 = 0.36 Cum
```

for vertical posts.

And finally:

```
Total Quantity = 12.20 Cum
```

This is extremely important for the application.

The user should **not manually calculate these numbers**.

Instead, the application should provide something like:

```
Measurement Item
--------------------------------

Long Wall
No.         2
Length      6.66 m
Breadth     0.83 m
Depth       0.75 m

Quantity = automatically calculated
         = 8.29 Cum
```

Then:

```
Short Wall
No.         2
Length      2.85
Breadth     0.83
Depth       0.75

Quantity = 3.55 Cum
```

Then the system automatically totals everything.

---

5. The measurement engine is the first major automation

The application needs a generic quantity formula engine.

For a volume:

```
Quantity = Number × Length × Breadth × Depth
```

For area:

```
Quantity = Number × Length × Height
```

For running metre:

```
Quantity = Number × Length
```

For count-based items:

```
Quantity = Number
```

But there is another important operation:

Deductions

For masonry, plaster etc., the document deducts doors/windows.

Example from the Excel:

Gross masonry:

```
10.06 Cum
```

Deductions:

```
D1 = 0.79
D2 = 0.36
W1 = 0.41
W2 = 0.56
```

Total deduction:

```
2.12 Cum
```

Net:

```
10.06 - 2.12
= 7.94 Cum
```

Your application therefore needs a proper concept of:

```
Gross Quantity
-
Deduction Quantity
=
Net Quantity
```

This cannot just be a simple calculator.

---

6. The `AB` sheet is the Abstract Estimate

Once the measurement quantities are calculated, they are transferred into the **`AB`** sheet.

The Excel contains 18 estimate items.

For example:

Item	Quantity	Rate	Unit	Amount
Excavation	12.20	120	Cum	1,464
Rubble filling	4.88	905	Cum	4,416
CC 1:4:8	2.44	3476.10	Cum	8,482
UCR masonry	3.59	1797.35	Cum	6,452
Brick masonry	7.94	2585.20	Cum	20,526
Teak roofing members	0.89	86131.60	Cum	76,657
Internal plaster	32.41	150.75	Sqm	4,886
Colour wash	32.41	15	Sqm	486
Sand faced plaster	15.60	362.15	Sqm	5,650
CGI roofing	73.01	650.05	Sqm	47,460
Flooring	19.32	313.60	Sqm	6,059
Jingle wood ballies	50.60	110	Rmt	5,566
GI pipe	19.80	320.65	Rmt	6,349
Grill gate	1.95	1903.40	Sqm	3,712
Kud roof/wall	37.84	73.25	Sqm	2,772
Point wiring	7	410	No.	2,870
Teak frame	0.12	96131.60	Cum	11,536
Double leaf doors	13.23	3501.60	Sqm	46,326

The workbook's total is:

```
₹261,669
```

The calculation is basically:

```
Amount = Quantity × Rate
```

with rounding.

So one major part of your software is:

```
Measurement Engine
        ↓
Quantity
        ↓
Rate Lookup
        ↓
Amount
        ↓
Abstract Total
```

7. `Sheet1` is effectively a Rate Database

This is very important.

`Sheet1` contains hundreds of construction items and their corresponding rates.

It references sources such as:

- PWD CSR 2014-15
- WRD CSR 2013-14
- other departmental schedule references

For example, the workbook contains:

```
Item 1
Excavation
Rate = 120
Unit = Cum
```

```
Item 3
Rubble filling
Rate = 905
```

```
Item 4
Cement concrete
Rate = 3476.10
```

```
Item 72
Country teak wood roofing members
Rate = 86131.60
Unit = Cum
```

```
Item 73
GI pipe
Rate = 320.65
Unit = Rmt
```

```
Item 87
Colour wash
Rate = 15
Unit = Sqm
```

```
Item 95.1
Point Wiring
Rate = 410
Unit = No.
```

Therefore the application should **not hardcode rates into the calculation code**.

It should have a database:

```
Rate Schedule
---------------------------
Item Code
Description
Rate
Unit
Schedule Source
Schedule Year
Department
Effective From
Effective To
```

Then:

```
Measurement Item
        ↓
Select Item Code
```

```
        ↓
Fetch Current Applicable Rate
        ↓
Calculate Amount
```

8. This is where your application can dramatically improve the Excel process

Currently they have formulas like:

```excel
=VLOOKUP(...)
```

everywhere.

The application can replace that with a database query:

```
SELECT rate, unit
FROM rate_items
WHERE item_code = ?
AND schedule_year = ?
```

Much safer and easier to maintain.

---

9. Depreciation is a separate calculation engine

The `Dep` sheet contains:

```
Type of structure
Total life of structure
Present Estimated Cost
Year of construction
Present life
Future life
Y.P. for future life @ 7%
Y.P. for total life @ 7%
Depreciated Value
```

For this case:

```
Present Estimated Cost = ₹261,669

Total life = 45 years

Year of construction = 2012

Present life = 4 years

Future life = 41 years
```

The workbook derives:

```
Y.P. future life @ 7% = 13.394
Y.P. total life @ 7%   = 13.606
```

Then:

```
Depreciated Value
=
Present Estimated Cost
×
Y.P. Future Life
÷
Y.P. Total Life
```

So:

```
₹261,669 × 13.394 / 13.606
≈ ₹257,592
```

The application needs to implement this formula exactly according to the approved methodology.

---

10. `Y.P.` is a lookup table, not random numbers

The `Y.P.` sheet contains Year/Present-value-style factors.

For example:

```
1 → 0.935
2 → 1.808
3 → 2.624
...
45 → 13.606
```

The depreciation sheet uses this table with:

```excel
VLOOKUP(...)
```

So in the application we should store:

```
Depreciation Factors
--------------------
Year
YP Factor @ 7%
```

Then the calculation engine can retrieve the appropriate factor.

This should be configurable instead of embedded permanently into source code.

---

11. There is a second valuation

This is one of the things that makes the project more interesting.

The workbook contains:

```
AB 10%
```

This is labelled:

```
ABSTRACT (salvage)
```

It contains a subset of items considered in the second valuation.

For this case the selected items include things such as:

- teak wood roofing components
- CGI sheets
- GI pipe
- grill gate
- teak wood frame
- double leaf doors

The total is:

```
₹192,040
```

Then this goes into:

```
Dep (2)
```

where it is depreciated.

The calculated depreciation value is:

```
₹183,981
```

Then the workbook performs:

```
Depreciation Amount 1
-
10% of Depreciation Amount 2
```

Specifically:

```
₹257,592
-
₹18,398.10
=
₹239,193.90
```

And this value eventually reaches the `FC` sheet.

This is one of the most important business rules your team needs to verify with the domain expert before implementation.

Do not assume that this rule is universally correct just because it exists in this spreadsheet.

The software should implement the official approved rule, not merely reproduce a potentially old spreadsheet.

---

12. `FC` is the final estimate/cover sheet

The `FC` sheet is basically the formal government-facing estimate document.

It pulls:

- owner
- house number
- village
- L.A. case number
- final amount
- project information
- department information

from other sheets.

The workbook currently produces:

```
₹239,193.90
```

as the final amount.

So conceptually:

```
Measurement
     ↓
Abstract
₹261,669
     ↓
Depreciation 1
₹257,592
     ↓
Salvage Abstract
₹192,040
     ↓
Depreciation 2
₹183,981
     ↓
10% adjustment
₹18,398.10
     ↓
Final
₹239,193.90
```

13. Important: the PDF and Excel don't perfectly match

This is a major finding.

The PDF has older/different handwritten/printed calculations.

For example, pages 3–4 show values around:

```
₹280,xxx
₹197,xxx
and final calculations around ₹255,xxx
```

while the Excel workbook currently computes:

```
₹261,669
→ ₹257,592
→ ₹192,040
```

```
→ ₹183,981
→ ₹239,193.90
```

Therefore:

Do not build the application by assuming the PDF numbers and Excel numbers represent one perfectly synchronized dataset.

There are clearly different versions/calculation states.

This actually makes your project more important because the application should provide:

```
Input data
+
rate schedule/version
+
calculation rules/version
+
audit trail
=
reproducible estimate
```

rather than somebody editing cells manually.

---

14. `Final RA` is another important discovery

`Final RA` means Rate Analysis.

It contains hundreds of formulas that refer to another workbook/file:

```
'[5]Rates Only'!
```

This means the current Excel system is **not completely self-contained**.

It depends on external Excel files.

That is a serious weakness.

Your application should eliminate this dependency.

Instead of:

```
Excel file
  ↓
another Excel
  ↓
another Excel
  ↓
VLOOKUP
```

you should have:

```
Application Database
        ↓
Rate Schedule
        ↓
Versioned Rate Items
```

15. `Sheet2`

Sheet2 appears to be another rate/item lookup table.

It contains item numbers, descriptions, units and rates, again using lookups into other sheets/files.

This looks more like an intermediate/legacy worksheet than the main user-facing calculation sheet.

For your application, I would **not reproduce this sheet literally**.

Instead, normalize its useful information into the rate database.

---

16. What the person who gave you this idea probably expects

Based on the documents, I would interpret the project requirement as:

> Build a system that takes property/building information and construction measurements, automatically identifies construction work items, calculates quantities, applies the appropriate government schedule rates, prepares the abstract estimate, performs depreciation/salvage calculations according to the approved methodology, and generates the final valuation document.

That is much clearer than simply saying:

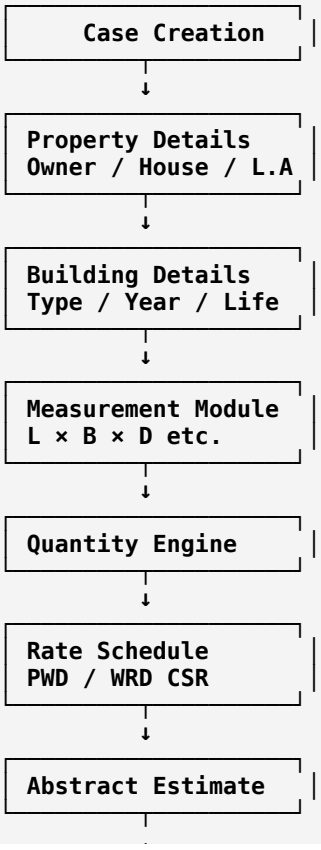
> "Automate estimation calculation."

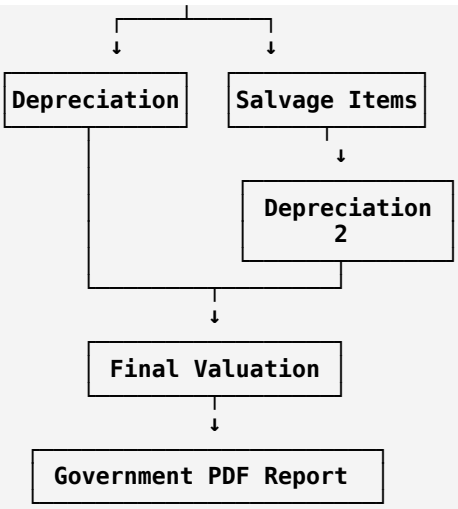
---

17. The proposed application

I would design it as a **House Valuation & Estimation Automation System**.

The architecture should look like:





18. The user interface should NOT look like Excel

Do not make:

```
Excel in browser
```

That would only move the problem.

Instead, create a guided workflow.

Step 1 — Create Case

```
Project
Village
Taluka
District
L.A. Case Number
Owner Name
House Number
Survey/Gat Number
Date
```

Step 2 — Property Details

```
Structure Type
Year of Construction
Present Life
Total Life
Built-up / Plinth Area
Number of Rooms
Roof Type
Wall Type
Floor Type
```

19. Step 3 — Measurement Builder

This should be the heart of the application.

Example:

```
Construction Item
[ Excavation ]

Calculation Type
[ Volume ]

Long Wall
No.       [2]
Length    [6.66]
Breadth   [0.83]
Depth     [0.75]

                          Quantity
                          8.29 Cum
```

Then:

```
+ Add Measurement
```

The user can add:

```
Short Wall
Vertical Posts
Steps
Room
Door Deduction
Window Deduction
```

The system calculates everything.

---

20. Template-based measurement will make the system much more powerful

This is where I'd take the project beyond a simple calculator.

Instead of the engineer manually building every formula, create templates:

```
Foundation Excavation
Wall Masonry
Plaster
Roofing
Flooring
Door
Window
Electrical
Wood Work
```

Each template contains:

```
Input dimensions
Formula
Unit
Deduction rules
Applicable rate items
```

For example:

```
Wall Volume Template
```

```
Quantity =
Number × Length × Thickness × Height
```

and:

```
Wall Net Quantity =
Gross Wall Quantity
-
Door Deductions
-
Window Deductions
```

This converts the application into a reusable **estimation engine**.

---

21. Step 4 — Automatic rate selection

Once the engineer chooses:

```
Item:
Country teak wood common rafters
```

the system automatically shows:

```
Description
Rate: ₹86,131.60
Unit: Cum
Schedule: PWD CSR
Year: 2014-15
```

The engineer should not type ₹86,131.60 manually.

This prevents one of the biggest sources of Excel mistakes.

---

22. Step 5 — Abstract estimate

The application generates:

```
Item No.
Description
Quantity
Rate
Unit
Amount
```

with:

```
Total = Σ(quantity × rate)
```

And the user can inspect every line.

For example:

```
1  Excavation             12.20   120       1,464
2  Rubble Filling          4.88   905       4,416
3  CC 1:4:8                2.44   3476.10   8,482
...
```

```
18 Double Leaf Door       13.23   3501.60  46,326

                        ------------------------
                          ₹261,669
```

23. Step 6 — Depreciation calculator

The user enters:

```
Construction Year
2012

Total Life
45

Valuation Date
[date]
```

The application calculates:

```
Present Life
Future Life
YP Future Life
YP Total Life
Depreciated Value
```

No manual VLOOKUP.

---

24. Step 7 — Salvage/second valuation

The application should allow the engineer to mark:

```
Include in Salvage Calculation
✓
```

against relevant items.

For example:

```
Item                          Salvage?
Teak Roofing                  ✓
CGI Roofing                   ✓
GI Pipe                       ✓
Grill Gate                    ✓
Teak Frame                    ✓
Door                          ✓
Internal Plaster              ✗
Excavation                    ✗
```

Then automatically generate the second abstract.

This is much better than maintaining a separate Excel sheet called `AB 10%`.

---

25. Step 8 — Final valuation

The system should show the calculation transparently:

```
Original Abstract                    ₹261,669
Depreciated Value                    ₹257,592

Salvage Abstract                     ₹192,040
Salvage Depreciated Value            ₹183,981

Adjustment                            ₹18,398.10

-----------------------------------------------
Final Valuation                       ₹239,193.90
```

More importantly, every figure should be clickable:

```
₹257,592
   ↓
show calculation
```

This is what makes the system auditable.

---

26. Step 9 — Documents/evidence

The PDF contains photographs, Panchanama and drawing.

Your application should allow:

```
Upload Property Photo
Upload Measurement Drawing
Upload Panchanama
Upload Ownership Document
Upload Supporting Document
```

Then associate them with the case.

So one case becomes:

```
CASE-2026-000165

├── Property Details
├── Measurements
├── Rate Schedule
├── Abstract
├── Depreciation
├── Salvage
├── Final Valuation
├── Photos
├── Panchanama
├── Drawing
└── Generated Estimate PDF
```

27. Add an audit trail

This is extremely important for government/financial estimation.

Suppose someone changes:

```
Wall length
6.66 → 6.80
```

The system should record:

```
Changed by: Engineer X
Old value: 6.66
New value: 6.80
Date/time: ...
Reason: ...
```

Likewise for:

```
Rate
Quantity
Depreciation rule
Construction year
Salvage selection
```

This is something Excel does poorly.

---

28. The real MVP

Do **not** start by building AI.

Your first working version should only automate the deterministic calculations.

MVP

```
1. User login
2. Create valuation case
3. Property details
4. Measurement entry
5. Quantity calculation
6. Rate database
7. Abstract calculation
8. Depreciation calculation
9. Salvage calculation
10. Final valuation
11. PDF generation
12. Case history
```

Once this works correctly, then add advanced features.

---

29. Where AI can actually help

AI is useful later.

For example, the engineer could upload a drawing and say:

> Extract the dimensions from this building drawing.

AI/computer vision could propose:

```
Building length = 6.66m
Building width = 3.45m
Wall thickness = 0.23m
Height = 2.30m
```

But **the engineer must confirm them**.

Another possible feature:

```
Upload old estimate PDF
        ↓
OCR/document extraction
        ↓
Detect:
Owner
House number
Measurements
Items
Rates
        ↓
Draft case
        ↓
Engineer verifies
        ↓
Calculate
```

Do not make AI responsible for final financial calculations. The calculation engine should remain deterministic.

---

30. Recommended technology architecture

For your team, I would build:

```
Frontend:
React / Next.js

Backend:
Spring Boot Java

Database:
PostgreSQL

Authentication:
JWT / OAuth

File Storage:
S3-compatible storage

PDF:
Server-side PDF generation

Calculation Engine:
Java service

Deployment:
Docker
+
AWS EC2 / similar
```

You could also use a simpler Node backend, but because this is a calculation-heavy enterprise-style application and you are comfortable with Java, **Spring Boot + PostgreSQL is a strong choice**.

---

31. Database design

At minimum:

```
users
projects
cases
properties
structures
```

```
measurements
measurement_lines
deductions
rate_schedules
rate_items
estimate_items
abstracts
depreciation_rules
depreciation_factors
salvage_items
valuations
documents
audit_logs
```

A simplified relationship:

```
Project
   │
   └── Case
         │
         ├── Property
         │
         ├── Measurements
         │       │
         │       ├── Measurement Lines
         │       └── Deductions
         │
         ├── Abstract
         │       │
         │       └── Estimate Items
         │
         ├── Depreciation
         │
         ├── Salvage
         │
         ├── Final Valuation
         │
         └── Documents
```

32. One extremely important design decision

Do not store formulas like:

```
=VLOOKUP(...)
```

in your application.

Instead define actual business rules.

For example:

```Java
quantity = number * length * breadth * depth;
amount = round(quantity * rate);
```

For a wall:

```Java
grossQuantity =
        number * length * thickness * height;

netQuantity =
        grossQuantity - deductions;
```

Then depreciation:

```java
depreciatedValue =
        presentCost
        * futureLifeYP
        / totalLifeYP;
```

The result should be mathematically reproducible independently of the UI.

---

33. One more major issue: rate versioning

The PDF explicitly says that particular historical PWD/WRD schedule years were used.

Therefore the database needs:

```
PWD CSR 2014-15
WRD CSR 2013-14
...
```

as different versions.

Suppose today the rate of an item is:

```
₹5,800
```

but the old case requires:

```
₹5,100
```

The system must still be able to reproduce the old valuation.

So:

```
Rate Item
   │
   ├── 2013-14
   ├── 2014-15
   ├── 2015-16
   └── ...
```

This is essential.

---

34. The project can eventually support multiple departments/projects

Once you build the engine properly:

```
Estimation Platform
        │
        ├── Jigaon Project
        ├── Other Irrigation Project
        ├── Road Acquisition
```

```
        ├── Government Building Valuation
        └── Other Property Valuation
```

The calculation engine stays common while:

```
Project
Rate Schedule
Valuation Rules
Document Template
```

can differ.

That gives you an actual reusable product instead of one application hardcoded for this one Excel file.

---

35. What I think your team should do next

Before writing code, spend the first phase on **reverse-engineering the existing manual process**.

Your team should sit with the person who gave you this problem and get definitive answers to:

A. What is the actual input?

Do they start from:

```
House visit
+
drawing
+
measurements
+
photos
```

or do they receive an existing document?

B. Who enters measurements?

```
Junior Engineer?
Estimator?
Clerk?
Surveyor?
```

C. How are construction items selected?

Does the engineer manually choose:

```
Excavation
Brick masonry
Plaster
Roofing
Door
Window
...
```

or is there a standard checklist?

D. Which rate schedule is applicable?

This is critical.

E. How is depreciation officially calculated?

Especially:

```
7%
Y.P.
Total life
Future life
Salvage
10% adjustment
```

We should verify the official rule instead of blindly copying Excel.

F. What final document must be generated?

Does the final PDF need to visually reproduce:

```
FC
AB
Dep
AB 10%
Panchanama
```

or only generate a new official report?

---

36. The best project statement

From these documents, I would frame your project as:

> **Automated House Valuation and Estimation Management System for Government Acquisition/Submergence Projects**

The core goal:

> Convert the existing manual Excel-based measurement, rate lookup, abstract preparation, depreciation, salvage valuation and final estimate workflow into a centralized, auditable web application.

That is a much stronger project than:

> “We are making an estimation website.”

---

37. The architecture I would actually build

```
                FRONTEND
                   |
            React / Next.js
                   |
                   v
             REST API
                   |
            Spring Boot
                   |
```

```
        ┌──────────────┼──────────────┐
        ↓              ↓              ↓
Measurement      Rate Engine    Valuation Engine
Engine            Service          Service
        │              │              │
        └──────────────┼──────────────┘
                       ↓
                  PostgreSQL
                       │
              ┌────────┴────────┐
              ↓                 ↓
        Document Storage     Audit Logs
              │
              ↓
        PDF Report Engine
```

And later:

```
AI Document Extraction
        ↓
Draft Measurements
        ↓
Human Verification
        ↓
Deterministic Calculation Engine
```

Bottom line

After reading both documents, I don't think the original idea is simply “automate an estimate calculator.”

The real problem is:

An engineer currently has to manually collect property information, interpret a building drawing, prepare measurements, calculate quantities, select hundreds of possible government schedule items/rates, create an abstract, perform two valuation/depreciation stages, apply adjustments, prepare a final estimate and maintain supporting documents. The Excel workbook automates only part of this and itself depends on formulas and external/legacy workbooks.

That is exactly what your application should solve.

The strongest MVP is therefore:

Case → Property → Measurements → Automatic Quantity → Rate Selection → Abstract → Depreciation → Salvage → Final Valuation → Evidence → Government PDF → Audit Trail.

And the PDF should be treated as the **reference/output format**, while the Excel should be treated as the **existing calculation/business-process implementation**. The discrepancies between the PDF and Excel mean you should validate the calculation rules with the domain expert before locking the software logic.

मोहन विश्वनाथ गई १६०

ok to design the protodype i want a detailed descriptive propmt a very very detiled implemnetaion plan to desin the website which will take the inputs nd that kind of stuff and do create in last the required output pdf and that form a complte system propmt detiled which i can give to ai model or agent which will do implment in one by one

Show moreShow less

Below is a **master implementation prompt** designed to be given directly to a coding AI/agent such as Cursor, Claude Code, Codex, or another autonomous software-development agent.

It is deliberately structured so the agent builds the system **phase-by-phase**, rather than trying to generate the entire application in one uncontrolled step.

The specification below is based on the uploaded valuation PDF and Excel workbook. The important source structure is the measurement → abstract → depreciation → second/salvage valuation → final valuation workflow shown in the documents. The PDF contains 22 pages including estimate sheets, measurement sheets, photographs, Panchanama pages and a construction drawing.

मोहन विश्वनाथ गई १६०

MASTER IMPLEMENTATION PROMPT

Automated House Valuation & Estimation Management System

You are an expert full-stack software architect, UI/UX designer, backend engineer, database architect, calculation-engine developer, document-generation engineer, and QA engineer.

Your task is to design and implement a complete working prototype of a web-based **Automated House Valuation & Estimation Management System**.

The system must transform the current manual/Excel-driven valuation process into a structured digital workflow.

The system must accept property/project information, building measurements, construction items, deductions, applicable rate schedules and depreciation information, automatically calculate quantities and amounts, generate the abstract estimate, perform the required depreciation and second/salvage valuation calculations, calculate the final valuation and finally generate a professional PDF report.

IMPORTANT:

Do not treat this as a simple calculator.

This is a complete workflow-management and calculation application.

The system must have:

1. Case creation
2. Property information
3. Building information
4. Measurement entry
5. Automatic quantity calculation
6. Deduction handling
7. Construction item selection
8. Rate schedule lookup
9. Abstract estimate generation

10. Depreciation calculation
11. Salvage/second valuation
12. Final valuation
13. Supporting document uploads
14. Calculation history/audit trail
15. PDF generation
16. Dashboard/case management
17. Validation and error handling
18. A clean professional user interface

---

1. SOURCE MATERIAL CONTEXT

The system is based on an existing government-style house valuation workflow.

The provided reference documents contain:

- project information
- owner information
- house number
- L.A. case information
- construction details
- construction type
- estimated construction cost
- year of construction
- present life
- future life
- total life
- depreciation factors
- measurement sheets
- abstract estimate
- construction item rates
- second/salvage abstract
- second depreciation calculation
- photographs
- Panchanama information
- construction drawing

The PDF reference contains 22 pages. Pages include formal estimate information, depreciation calculations, abstract estimate tables, measurement calculations, property photographs, Panchanama documents and a construction drawing.

The Excel workbook contains sheets corresponding to major calculation stages, including:

- Cover Page
- FC
- Dep (2)
- AB 10%
- Dep
- AB
- mea
- Final RA
- Y.P.
- Sheet1
- Sheet2

These sheets represent the current calculation workflow.

Do not blindly reproduce Excel UI.

The purpose is to replace the spreadsheet workflow with a proper application architecture.

---

2. PRIMARY BUSINESS WORKFLOW

Implement the following workflow exactly as the core product flow:

PROJECT
↓
CASE
↓
PROPERTY DETAILS
↓
BUILDING / STRUCTURE DETAILS
↓
MEASUREMENT ENTRY
↓
QUANTITY CALCULATION
↓
DEDUCTIONS
↓
CONSTRUCTION ITEM SELECTION
↓
RATE LOOKUP
↓

ABSTRACT ESTIMATE
↓
DEPRECIATION
↓
SALVAGE / SECOND ABSTRACT
↓
SECOND DEPRECIATION
↓
FINAL VALUATION
↓
DOCUMENTS / EVIDENCE
↓
FINAL PDF REPORT

The user should be able to move through this process step-by-step.

---

3. IMPORTANT CALCULATION PRINCIPLE

Separate the system into:

A. UI layer

B. API/backend layer

C. Calculation engine

D. Rate engine

E. Document/PDF engine

F. Persistence/database

Never put important calculation logic directly into React components.

Never calculate final financial values only on the frontend.

The backend must independently calculate and validate all values.

The frontend only displays calculations returned by the backend and may provide instant preview calculations for UX.

The backend calculation result is authoritative.

---

4. PROTOTYPE GOAL

The first version is a functional prototype.

The prototype must demonstrate the complete end-to-end flow.

A user should be able to:

1. Open dashboard
2. Create a project
3. Create a valuation case

4. Enter owner/property information
5. Enter structure information
6. Add measurement items
7. Enter dimensions
8. Automatically calculate quantities
9. Apply deductions
10. Select construction items
11. Automatically retrieve rates
12. Generate abstract estimate
13. Calculate depreciation
14. Select salvage items
15. Generate second abstract
16. Calculate second depreciation
17. Calculate final valuation
18. Upload supporting documents
19. Preview the entire valuation
20. Generate PDF
21. Download/view generated PDF
22. Re-open the case and see all saved calculations

The prototype must use a seeded example case derived from the supplied Excel/PDF for demonstration.

Do not hardcode those values into the calculation engine.

Seed them into the database as normal records.

---

5. RECOMMENDED TECHNOLOGY STACK

Use this architecture unless there is a strong technical reason to change it.

Frontend

React + TypeScript

Preferred:

Next.js or Vite + React.

Use:

- TypeScript
- React

- Tailwind CSS
- shadcn/ui or equivalent professional component library
- React Hook Form
- Zod validation
- TanStack Query
- Zustand or equivalent lightweight state management where required
- Recharts for dashboard analytics if needed

Backend

Java + Spring Boot

Use:

- Spring Boot
- Spring Web
- Spring Data JPA
- Hibernate
- Bean Validation
- PostgreSQL driver
- Lombok only where useful
- JWT authentication for prototype authentication

Database

PostgreSQL

File storage

For prototype:

local storage directory or S3-compatible abstraction.

Design storage behind an interface so it can later be moved to:

AWS S3

PDF

Use a server-side PDF generation library.

The PDF generation must be deterministic.

Deployment

Use Docker.

Prepare:

- frontend container
- backend container
- PostgreSQL container

Use docker-compose for local development.

---

6. SYSTEM ARCHITECTURE

Use this logical architecture:

Frontend
↓
REST API
↓
Authentication
↓
Case Service
↓
Measurement Service
↓
Calculation Engine
↓
Rate Engine
↓
Valuation Engine
↓
Document Service
↓
PostgreSQL
↓
File Storage

Separate responsibilities.

Recommended backend packages:

com.application.auth

com.application.project

com.application.case

com.application.property

com.application.measurement

com.application.rate

com.application.estimate

com.application.depreciation

com.application.salvage

com.application.valuation

com.application.document

com.application.audit

com.application.pdf

com.application.common

---

7. USER ROLES

For the prototype implement:

ADMIN

Can:

- create project
- create case
- edit case
- manage rates
- manage rate schedules
- manage depreciation factors
- review calculations
- generate reports
- view all cases

ENGINEER / ESTIMATOR

Can:

- create case
- enter property information
- enter measurements
- select items
- calculate estimate
- calculate depreciation

- select salvage items
- generate PDF
- upload supporting documents

Do not overcomplicate permissions in the first prototype.

The architecture should allow additional roles later.

---

8. DASHBOARD

Create a professional dashboard.

Show:

Total Projects

Total Cases

Draft Cases

In Progress

Completed

Reports Generated

Recent Cases

Recent Valuations

Total Estimated Value

The dashboard must not use fake random numbers after the seeded sample.

Use actual database values.

---

9. PROJECT MANAGEMENT

A project represents a larger government/project context.

Fields:

projectId

projectName

department

division

district

taluka

description

projectCode

status

createdAt

updatedAt

Example from source context:

Jigaon Project

Do not hardcode Jigaon into the application.

It should merely be seeded as one example project.

---

10. CASE CREATION

Every valuation must belong to a case.

Create a case number.

Fields:

caseId

caseNumber

projectId

ownerName

houseNumber

village

taluka

district

LACaseNumber

surveyNumber

dateOfInspection

valuationDate

status

createdBy

createdAt

updatedAt

Example source case information should be seed data only.

---

11. PROPERTY INFORMATION PAGE

Create a clean form.

Sections:

Owner Information

Owner name

Contact number

Address

Village

Taluka

District

Legal/Case Information

L.A. Case Number

House Number

Survey/Gat Number

Project

Department

Division

Inspection Information

Inspection date

Valuation date

Prepared by

Checked by

Approved by

---

12. STRUCTURE INFORMATION

Fields:

Structure type

Construction type

Year of construction

Total useful life

Present life

Future life

Roof type

Wall type

Floor type

Number of rooms

Built-up area

Plinth area

Additional notes

The source document includes examples such as:

B.B.M. Wall + C.G.I. Sheet Roof

Do not hardcode this as the only option.

Provide selectable options plus custom entry.

---

13. MEASUREMENT ENGINE

This is the most important part of the application.

The measurement engine must support different calculation types.

At minimum:

VOLUME

Formula:

quantity = number × length × breadth × depth

Unit:

Cum

AREA

Formula:

quantity = number × length × height

Unit:

Sqm

RUNNING LENGTH

Formula:

quantity = number × length

Unit:

Rmt

COUNT

Formula:

quantity = number

Unit:

No.

CUSTOM FORMULA

Allow future custom formulas.

---

14. MEASUREMENT UI

Create a measurement builder.

Example:

Construction Item:

[Excavation]

Calculation Type:

[Volume]

Number:

[2]

Length:

[6.66]

Breadth:

[0.83]

Depth:

[0.75]

Calculated Quantity:

8.29 Cum

The quantity must update immediately for UX.

But when saving, send the dimensions to backend.

Backend must calculate the official quantity.

Display:

Gross Quantity

Deductions

Net Quantity

---

15. MEASUREMENT GROUPS

Measurements should be organized by construction item.

Example:

Excavation

Rubble Masonry

Brick Masonry

Roofing

Plaster

Flooring

Doors

Windows

Electrical

Wood Work

Other

The user can add unlimited measurement groups.

---

16. DEDUCTION ENGINE

The system must support deductions.

For each measurement item allow:

Add Deduction

Deduction Type:

Door

Window

Opening

Other

Example:

Gross quantity:

10.06 Cum

Deductions:

D1 = 0.79

D2 = 0.36

W1 = 0.41

W2 = 0.56

Total deduction:

2.12 Cum

Net quantity:

7.94 Cum

The exact formulas should be derived from the input dimensions.

Do not allow the user to manually type net quantity unless an explicit override capability is provided to an authorized role.

---

17. MEASUREMENT HISTORY

Every measurement must contain:

id

caseId

constructionItemId

calculationType

number

length

breadth

depth

height

grossQuantity

deductionQuantity

netQuantity

unit

notes

createdBy

updatedBy

createdAt

updatedAt

---

18. RATE MANAGEMENT

Create an admin rate-management page.

Fields:

rateId

scheduleId

itemCode

itemNumber

description

rate

unit

department

scheduleName

scheduleYear

referenceSource

effectiveFrom

effectiveTo

status

The source workbook clearly contains item numbers, descriptions, units and rates and references government schedule sources.

Do not hardcode the rates into Java classes.

---

19. RATE SCHEDULES

Create entities:

RateSchedule

RateItem

Example:

Rate Schedule:

PWD CSR

Year:

2014-15

Department:

Water Resources / relevant department

Rate Item:

Item Number

72

Description:

Country teak wood common rafters...

Rate:

86131.60

Unit:

Cum

The exact text imported from the reference workbook should be stored as data.

20. RATE SELECTION UX

When the engineer adds an estimate item:

Search:

[teak wood]

Show:

Item code

Description

Rate

Unit

Schedule

Year

Source

Select item.

Once selected:

Rate becomes visible but editable only if the user has permission.

Default behavior:

quantity × approved rate

21. ESTIMATE ITEM

Entity:

EstimateItem

Fields:

id

caseId

constructionItemId

measurementId

quantity

rate

unit

amount

salvageEligible

notes

sequence

createdAt

updatedAt

Amount formula:

amount = quantity × rate

Use controlled decimal precision.

Avoid floating-point precision errors.

Use BigDecimal in Java.

Never use double for monetary calculations.

---

22. ABSTRACT ESTIMATE

Create an abstract estimate page.

Columns:

Sr. No.

Item No.

Description

Quantity

Rate

Unit

Amount

Reference

The user should be able to expand an item and see the measurement that generated the quantity.

Example:

Item 5

Quantity:

0.89 Cum

Rate:

₹86,131.60

Amount:

₹76,657.xx

The source document shows this type of relationship between measurement sheets and abstract items.

---

23. ABSTRACT TOTAL

At bottom:

Subtotal

Adjustments if any

Grand Total

Use backend-calculated values.

Show a calculation summary panel.

---

24. DEPRECIATION ENGINE

Create a dedicated depreciation module.

Inputs:

Present Estimated Cost

Year of Construction

Valuation Date

Present Life

Future Life

Total Life

Applicable depreciation/YP method

The source workbook uses Year/Present-value-style lookup factors and calculates depreciation from present estimated cost and Y.P. factors.

For the prototype implement the source workbook's methodology only after confirming the official calculation rule.

Do not silently reinterpret it.

---

25. Y.P. FACTOR TABLE

Create:

DepreciationFactor

Fields:

id

year

rate

factor

scheduleType

effectiveFrom

effectiveTo

The source workbook contains a Y.P. lookup table.

Store these as database records.

Do not hardcode VLOOKUP logic into application code.

The Java calculation service should perform a lookup against the database.

---

26. DEPRECIATION CALCULATION

Conceptually implement:

Depreciated Value =

Present Estimated Cost

×

Y.P. for Future Life

÷

Y.P. for Total Life

All calculations must use BigDecimal.

Display:

Present Estimated Cost

Future Life

Y.P. Future Life

Total Life

Y.P. Total Life

Depreciation Factor

Depreciated Value

Show a “View Calculation” option.

---

27. SALVAGE / SECOND VALUATION

Implement a second abstract stage.

Each estimate item should have:

salvageEligible

The engineer can select items for the second valuation.

Example selectable items may include:

Roofing components

GI pipes

Grill gate

Wood frames

Doors

Other recoverable items

Do not hardcode which items must always be selected.

Store this as business configuration.

---

28. SECOND ABSTRACT

Generate:

Second Abstract / Salvage Abstract

Columns:

Sr. No.

Item

Quantity

Rate

Unit

Amount

Total

The reference workbook uses a separate “AB 10%” stage.

Do not call it “AB 10%” in the user interface unless there is a documented reason.

Use a human-friendly label:

“Salvage / Second Valuation”

Internally the database may map it to the legacy worksheet concept.

---

29. SECOND DEPRECIATION

Perform the second depreciation stage.

Inputs:

Second abstract total

Applicable construction life

Relevant Y.P. values

Output:

Second depreciated value

Display the calculation independently.

---

30. FINAL VALUATION ENGINE

The final valuation engine must combine:

Primary depreciated value

Second/salvage valuation

Approved adjustment rules

Any applicable deductions

Final valuation

Important:

The exact final adjustment formula currently visible in the workbook must be treated as a source-derived rule that requires domain confirmation before being declared universally valid.

Do not present a potentially workbook-specific formula as a universal government rule.

For the prototype, make the rule configurable.

Example configuration:

Adjustment type:

Percentage

Adjustment percentage:

10%

Adjustment basis:

Second depreciated value

Then calculate:

Adjustment Amount

Final Valuation

This allows the prototype to reproduce the provided case while keeping the rule configurable.

---

31. CALCULATION BREAKDOWN UI

Create a final calculation screen.

Example structure:

PRIMARY ESTIMATE

₹261,669

↓

PRIMARY DEPRECIATION

₹257,592

↓

SALVAGE ABSTRACT

₹192,040

↓

SALVAGE DEPRECIATION

₹183,981

↓

CONFIGURED ADJUSTMENT

₹18,398.10

↓

FINAL VALUATION

₹239,193.90

These numbers should come dynamically from the selected case.

Do not hardcode them in UI.

---

32. CALCULATION EXPLAINABILITY

Every important number should have:

[View Calculation]

When clicked show:

Input values

Formula

Intermediate values

Final result

Example:

Quantity

=

2 × 6.66 × 0.83 × 0.75

=

8.2953

Rounded:

8.30 Cum

This will make the system trustworthy.

---

33. CASE STATUS

Implement statuses:

DRAFT

MEASUREMENT_IN_PROGRESS

ESTIMATE_IN_PROGRESS

DEPRECIATION_IN_PROGRESS

REVIEW

APPROVED

COMPLETED

ARCHIVED

Users should clearly see status.

---

34. VALIDATION

Examples:

Cannot create valuation without owner name.

Cannot calculate estimate without measurements.

Cannot calculate estimate item without rate.

Cannot calculate depreciation without construction year.

Construction year cannot be in the future.

Total life must be greater than present life.

Future life cannot be negative.

Quantity cannot be negative.

Rate cannot be negative.

Monetary values cannot be NaN/infinite.

Required fields must be validated.

---

35. DOCUMENT MANAGEMENT

Allow upload:

Property photographs

Construction drawing

Panchanama

Ownership documents

Previous estimate

Other supporting documents

For each:

documentId

caseId

fileName

fileType

fileSize

storagePath

documentType

uploadedBy

uploadedAt

description

---

36. DOCUMENT PREVIEW

For images:

Display image preview.

For PDFs:

Display PDF preview.

For unsupported files:

show metadata.

Allow:

upload

replace

delete

download

preview

---

37. AUDIT LOG

Every important change must create an audit record.

Fields:

auditId

caseId

userId

action

entityType

entityId

oldValue

newValue

timestamp

reason

Examples:

RATE_CHANGED

MEASUREMENT_UPDATED

MEASUREMENT_DELETED

CASE_UPDATED

SALVAGE_SELECTION_CHANGED

DEPRECIATION_RECALCULATED

REPORT_GENERATED

This is extremely important.

---

38. VERSIONING

Create calculation versions.

Every time the final valuation is generated, save:

calculationVersion

rateScheduleVersion

depreciationFactorVersion

calculationTimestamp

calculatedBy

Input snapshot

Output snapshot

This ensures old reports can be reproduced later.

---

39. FINAL REVIEW SCREEN

Before generating PDF, show:

Case Summary

Owner

House

Village

Project

L.A. Case

Construction

Structure type

Year

Life

Measurements

Number of measurement items

Total quantities

Abstract

Number of items

Total

Depreciation

Primary value

Salvage

Second abstract

Second depreciation

Final

Final valuation

Buttons:

[Edit]

[Recalculate]

[Approve]

[Generate PDF]

---

40. PDF GENERATION

The PDF is one of the most important outputs.

Generate a professional multi-page PDF.

Do not simply print a browser screenshot.

The PDF should have structured sections.

Suggested structure:

PAGE 1

Cover / Case Information

PAGE 2

Formal Estimate Information

PAGE 3

Construction Details

PAGE 4

Primary Depreciation

PAGE 5+

Abstract Estimate

PAGE +

Measurement Sheet

PAGE +

Salvage / Second Abstract

PAGE +

Second Depreciation

PAGE +

Final Valuation

PAGE +

Supporting Documents

The exact pagination can evolve during implementation.

---

41. PDF CONTENT

Header:

Department

Project

Division

Section / Office

Document title

Case information

Owner information

Property information

---

42. PDF ESTIMATE TABLE

Include:

Quantity

Item No.

Particulars

Rate

Reference

Unit

Amount

Use proper table formatting.

Do not allow tables to overflow page boundaries.

Repeat column headers when tables continue onto another page.

---

43. PDF MEASUREMENT TABLE

Include:

Item

Description

Formula

Dimensions

Quantity

Unit

Deductions

Net Quantity

This makes the final report auditable.

---

44. PDF DEPRECIATION SECTION

Show:

Present Cost

Year of Construction

Present Life

Future Life

Total Life

Y.P. Future Life

Y.P. Total Life

Formula

Depreciated Value

45. PDF FINAL VALUATION

Show a prominent summary:

Primary Estimate

Primary Depreciated Value

Second/Salvage Estimate

Second Depreciated Value

Adjustment

Final Valuation

Use clear formatting.

46. SUPPORTING DOCUMENTS IN PDF

Allow optional appendices.

Examples:

Property photographs

Drawing

Panchanama

Supporting documents

The prototype can append images and supporting pages into the final report.

47. PDF TEMPLATE DESIGN

The generated document should look like an official professional engineering/valuation report.

Use:

A4

portrait/landscape depending on table

proper margins

page number

header

footer

case number

generated date

prepared by

checked by

approval placeholder

Do not copy government logos/seals unless they are supplied and authorized for use in the final prototype.

Use a neutral professional header for the prototype.

---

48. PDF OUTPUT NAME

Use:

CASE_<CASE_NUMBER>*VALUATION*<VERSION>.pdf

Example:

CASE_15-2008-09_VALUATION_V1.pdf

Sanitize file names.

---

49. EXCEL IMPORT

Implement Excel import only after the core workflow works.

The prototype should eventually support:

Upload existing workbook

Detect sheets

Map columns

Import rates

Import measurements

Import owner information

Show validation issues

Preview imported data

Confirm import

This feature should never silently overwrite existing records.

---

50. SAMPLE DATA

Create a seed dataset based on the provided reference case.

Use the source workbook values as seed records.

The goal is:

Open sample case

See data already populated

Click Recalculate

Get the same result as the source workbook wherever the source values/rules are explicitly represented.

Do not manually insert a fake final amount.

The final amount must emerge from calculations.

---

51. IMPORTANT SOURCE RECONCILIATION

There are differences between values visible in the PDF and values calculated/present in the Excel workbook.

Therefore:

Do not attempt to “correct” the documents automatically.

Do not silently assume one is wrong.

Instead:

1. Preserve the Excel calculation structure for prototype reproduction.
2. Preserve the PDF as the reference report layout.
3. Mark calculation rules requiring domain confirmation.
4. Make uncertain adjustment rules configurable.
5. Add a calculation audit trail.

---

52. UI PAGE STRUCTURE

Create the following routes/pages:

/login

/dashboard

/projects

/projects/new

/projects/

/cases

/cases/new

/cases/

/cases//property

/cases//structure

/cases//measurements

/cases//estimate

/cases//depreciation

/cases//salvage

/cases//final-valuation

/cases//documents

/cases//review

/cases//report

/admin/rates

/admin/rate-schedules

/admin/depreciation-factors

/admin/users

---

53. CASE WIZARD

Use a visible stepper:

1. Basic Information
2. Property
3. Structure
4. Measurements
5. Estimate
6. Depreciation
7. Salvage
8. Final Valuation
9. Documents
10. Review
11. Generate Report

Show completed/incomplete steps.

Do not allow accidental loss of data when navigating between steps.

Auto-save draft data where appropriate.

---

54. UI DESIGN PRINCIPLES

The website must look like a modern professional enterprise application.

Do not make it look like:

- a generic admin template
- a spreadsheet
- a student demo
- a plain CRUD application

Design principles:

clear hierarchy

large readable financial values

clean forms

minimal visual clutter

responsive tables

sticky totals

strong validation feedback

clear status indicators

consistent spacing

professional typography

desktop-first but responsive

---

55. MEASUREMENT UX DETAILS

For dimensions use numeric inputs.

Show units beside fields.

Example:

Length [6.66] m

Breadth [0.83] m

Depth [0.75] m

Never make users remember units.

When a formula is selected, dynamically show relevant fields.

For Area:

Number

Length

Height

For Volume:

Number

Length

Breadth

Depth

For Running Length:

Number

Length

For Count:

Number only

---

56. CALCULATION PREVIEW

Display live:

Formula

Gross Quantity

Deduction

Net Quantity

Rate

Amount

But clearly mark:

“Preview — final value calculated and validated by server.”

After saving/recalculating, show server-confirmed result.

---

57. SMART SEARCH

For construction items:

Search by:

item number

description

keyword

unit

schedule

year

Example:

user types:

“teak”

system returns relevant construction items.

---

58. TABLE FILTERING

Rate table must support:

search

department filter

schedule filter

year filter

unit filter

active/inactive

pagination

sorting

---

59. RATE ADMIN

Admin must be able to:

add rate

edit rate

deactivate rate

create schedule

import CSV/Excel

view source

view change history

Do not delete historically used rates physically.

Use inactive/archived status.

---

60. DATABASE MODEL

Use normalized tables.

Recommended initial schema:

users

roles

projects

valuation_cases

properties

structures

construction_items

measurement_groups

measurements

measurement_deductions

rate_schedules

rate_items

estimate_items

abstracts

depreciation_factors

depreciation_calculations

salvage_selections

salvage_estimates

valuation_adjustments

final_valuations

documents

audit_logs

calculation_versions

report_files

61. API DESIGN

Examples:

POST /api/auth/login

GET /api/projects

POST /api/projects

GET /api/cases

POST /api/cases

GET /api/cases/{id}

PUT /api/cases/{id}

POST /api/cases/{id}/measurements

PUT /api/measurements/{id}

DELETE /api/measurements/{id}

GET /api/rates

GET /api/rates/search

POST /api/cases/{id}/estimate/calculate

POST /api/cases/{id}/depreciation/calculate

POST /api/cases/{id}/salvage/calculate

POST /api/cases/{id}/final/calculate

POST /api/cases/{id}/documents

POST /api/cases/{id}/report/generate

GET /api/cases/{id}/report

62. CALCULATION ENGINE DESIGN

Create isolated calculation services.

Example:

MeasurementCalculationService

RateCalculationService

AbstractCalculationService

DepreciationCalculationService

SalvageCalculationService

FinalValuationService

ReportGenerationService

Do not combine all calculations into one large class.

---

63. BIGDECIMAL REQUIREMENT

All monetary calculations must use:

Java BigDecimal.

Examples:

rate

amount

valuation

depreciation

adjustment

Do not use float/double for financial amounts.

Define consistent rounding.

For example:

quantity precision configurable

money precision = 2 decimal places

The rounding policy must be centralized.

---

64. CALCULATION CONFIGURATION

Create:

CalculationConfiguration

Fields can include:

quantityDecimalPlaces

currencyDecimalPlaces

roundingMode

depreciationRate

adjustmentPercentage

allowedLifeRange

The configuration must be versioned.

---

65. ERROR HANDLING

Backend errors must be structured:

code

message

field

details

timestamp

Example:

MEASUREMENT_MISSING_LENGTH

INVALID_RATE

DEPRECIATION_FACTOR_NOT_FOUND

INVALID_CONSTRUCTION_YEAR

NEGATIVE_QUANTITY

CASE_NOT_READY_FOR_FINAL_VALUATION

The frontend must show human-readable error messages.

---

66. SECURITY

Use authentication.

Passwords must never be stored in plain text.

Use hashed passwords.

JWT expiration.

Role-based endpoint protection.

Validate all uploaded files.

Limit file sizes.

Prevent path traversal.

Do not expose internal storage paths.

Do not expose database credentials.

Use environment variables.

67. DO NOT BUILD THESE THINGS TOO EARLY

Do not initially implement:

complex AI

automatic drawing interpretation

OCR

mobile application

microservices

Kubernetes

complex cloud infrastructure

real government SSO

advanced predictive analytics

Build a strong modular monolith first.

68. FUTURE AI FEATURES

After the deterministic system works, possible AI extensions include:

PDF information extraction

OCR

drawing dimension extraction

construction-item suggestion

automatic classification of construction description

rate-item recommendation

anomaly detection

explanation of calculation differences

natural-language case search

Example:

User:

“Show houses where roofing cost exceeds 25% of the total valuation.”

AI translates the request into a structured query.

But AI must never silently alter official calculation values.

69. TESTING REQUIREMENTS

Write:

Unit tests

Integration tests

Calculation tests

API tests

Frontend component tests

End-to-end tests

PDF generation tests

70. CALCULATION TEST CASES

Test:

0 quantity

1 quantity

decimal dimensions

deductions greater than gross

zero rate

missing rate

negative dimensions

very large values

rounding

missing Y.P. factor

construction year equal to valuation year

future construction year

present life = total life

future life = 0

multiple deductions

multiple measurement items

multiple salvage items

71. GOLDEN TEST

The supplied example case must become a golden regression test.

For every major calculation:

Input

Expected output

Tolerance/precision

Store these expected values according to the validated source workbook rule.

If future code changes produce different values, the test should fail.

This prevents accidental changes to business logic.

---

72. PERFORMANCE

Prototype target:

Dashboard < 2 seconds under normal local conditions.

Case loading < 2 seconds.

Calculation < 1 second for normal case.

PDF generation should provide loading feedback.

Do not perform unnecessary database queries.

Use pagination for rate tables.

---

73. RESPONSIVE DESIGN

Desktop is primary.

Also support:

tablet

small laptop

mobile viewing

Large estimate tables should allow horizontal scrolling rather than destroying the layout.

---

74. EMPTY STATES

Every module needs proper empty states.

Examples:

“No measurements added.”

“No rate schedule selected.”

“No supporting documents uploaded.”

“Salvage items have not been selected.”

Do not display broken/blank UI.

---

75. USER CONFIRMATION BEFORE FINALIZATION

Before generating the official-looking report:

Show confirmation:

“All calculations have been reviewed. Generate valuation report?”

After generation:

Create immutable calculation version.

Do not overwrite the previous generated report.

---

76. REPORT VERSIONING

Example:

Version 1

Version 2

Version 3

If a measurement changes after Version 1:

Do not modify Version 1.

Create Version 2.

Display:

Previous Version

Current Version

Difference

---

77. CHANGE COMPARISON

For the prototype, implement a simple comparison.

Example:

Measurement:

6.66 → 6.80

Quantity:

8.29 → 8.47

Amount:

₹X → ₹Y

Final valuation:

₹A → ₹B

This is very useful for reviewing modifications.

---

78. IMPORTED LEGACY WORKBOOK HANDLING

If Excel import is implemented:

Do not assume every workbook has the same sheet names.

Show:

Detected sheets

Mapped sheets

Unmapped sheets

Validation warnings

Example:

Sheet found:

mea

Map to:

Measurement Sheet

Sheet found:

AB

Map to:

Abstract Estimate

Sheet found:

Y.P.

Map to:

Depreciation Factors

Sheet found:

Sheet1

Map to:

Rate Items

Require user confirmation.

---

79. NO DATA LOSS

Every important step should save independently.

Never wait until the final “Submit” button to save the whole case.

Use:

Save Draft

Next

Save & Continue

Auto-save where appropriate.

---

80. SEED DATA

Create:

1 sample project

1 sample case

sample construction items

sample rate schedule

sample Y.P. factors

sample measurements

sample deductions

sample estimate

sample salvage selection

sample valuation

The sample case should visually demonstrate the complete workflow.

---

81. DEVELOPMENT PHASES

Do NOT attempt to implement everything at once.

Implement in this exact order.

PHASE 0 — Project Analysis

Before writing code:

Analyze this specification.

Create:

architecture.md

database-design.md

calculation-rules.md

api-design.md

ui-flow.md

Do not start coding until this analysis is internally consistent.

PHASE 1 — Project Setup

Create:

frontend

backend

database

Docker setup

environment configuration

basic README

Git configuration

Health endpoint

Frontend/backend connection

Acceptance criteria:

Application starts locally.

Database connects.

Frontend calls backend successfully.

PHASE 2 — Authentication

Implement:

login

JWT

roles

protected routes

logout

Acceptance criteria:

Admin can log in.

Engineer can log in.

Protected pages cannot be accessed without authentication.

PHASE 3 — Dashboard

Implement:

dashboard

project list

case list

status cards

recent cases

Acceptance criteria:

No fake dashboard data.

Dashboard uses database.

PHASE 4 — Project + Case Management

Implement:

project CRUD

case creation

case list

case details

Acceptance criteria:

User can create a project.

User can create a case.

Case persists after refresh.

PHASE 5 — Property + Structure

Implement forms.

Validation.

Save/edit.

Acceptance criteria:

All property and structure data persists.

PHASE 6 — Measurement Engine

Implement:

calculation types

dimension entry

quantity calculations

deductions

server validation

Acceptance criteria:

Measurement quantity is correctly calculated.

Deductions correctly affect net quantity.

PHASE 7 — Rate Engine

Implement:

rate schedules

rate items

search

selection

admin management

Acceptance criteria:

Estimate items can retrieve correct rate.

Rate source/year is visible.

PHASE 8 — Abstract Estimate

Implement:

automatic estimate generation

item table

amount calculations

grand total

calculation details

Acceptance criteria:

Quantity × rate generates amount.

Total updates correctly.

PHASE 9 — Depreciation

Implement:

YP table

depreciation logic

calculation preview

calculation persistence

Acceptance criteria:

The result matches validated source calculations.

PHASE 10 — Salvage

Implement:

salvage item selection

second abstract

second depreciation

Acceptance criteria:

Second valuation works independently.

PHASE 11 — Final Valuation

Implement:

configured adjustment

final valuation

calculation breakdown

Acceptance criteria:

Final valuation is reproducible.

PHASE 12 — Documents

Implement:

uploads

preview

delete

download

Acceptance criteria:

Documents attach to the correct case.

PHASE 13 — PDF

Implement:

cover

case details

measurements

abstract

depreciation

salvage

final valuation

supporting images/documents

Acceptance criteria:

A clean A4 multi-page PDF is produced.

PHASE 14 — Audit + Versioning

Implement:

audit logs

calculation versions

report versions

Acceptance criteria:

Previous calculations remain reproducible.

PHASE 15 — Testing

Implement:

unit tests

API tests

calculation regression tests

golden sample test

PDF test

end-to-end test

PHASE 16 — UI POLISH

Improve:

spacing

responsive behavior

loading states

empty states

error messages

table UX

navigation

accessibility

visual consistency

82. AGENT EXECUTION RULES

You are an autonomous coding agent, but do not make uncontrolled changes.

For each phase:

1. Inspect existing project.
2. Understand current architecture.
3. Plan the phase.
4. Implement only that phase.
5. Run tests.
6. Fix errors.
7. Verify database migration.
8. Verify frontend/backend integration.
9. Update documentation.
10. Report what changed.
11. Only then proceed to the next phase.

Never jump directly from Phase 1 to Phase 10.

83. DO NOT DESTROY EXISTING WORK

If an existing repository is provided:

First inspect:

package.json

pom.xml

build.gradle

src/

database/

docker/

README

existing APIs

existing components

Do not rewrite the entire project unless explicitly necessary.

Reuse working components.

Preserve existing functionality.

84. CODE QUALITY REQUIREMENTS

Use:

clear names

small services

small components

DTOs

repositories

service layer

controller layer

validation

central exception handling

database migrations

environment configuration

typed API models

Meaningful comments only.

Do not create giant files containing unrelated functionality.

---

85. NO PLACEHOLDER IMPLEMENTATION

Do not write:

TODO: implement later

fake calculation

random number

hardcoded final valuation

fake database result

fake PDF

mock API permanently

If a feature cannot yet be implemented, clearly isolate it behind an interface and document it.

---

86. NO HARDCODED BUSINESS VALUES

Do not hardcode:

rates

YP factors

owner information

case numbers

final amount

adjustment percentage

construction items

project names

Instead:

Database

Configuration

Seed data

---

87. CALCULATION TRANSPARENCY

Every final amount must be traceable:

Final Valuation

↓

Adjustment

↓

Secondary Depreciation

↓

Secondary Abstract

↓

Primary Depreciated Value

↓

Primary Abstract

↓

Estimate Items

↓

Measurements

↓

Input Dimensions

↓

Rate Schedule

This traceability is a core product requirement.

---

88. SAMPLE USER JOURNEY

Demonstration scenario:

User logs in.

Dashboard appears.

User opens:

Jigaon Project

User opens sample case:

Mohan Vishwanath Gai

User opens Measurements.

The system displays measurement entries.

User changes one dimension.

System recalculates quantity.

User opens Estimate.

Amount changes automatically.

User opens Depreciation.

Depreciation recalculates.

User opens Salvage.

Selected items are shown.

User opens Final Valuation.

New final amount is shown.

User opens Review.

All calculations are summarized.

User clicks Generate PDF.

System creates Version 2 report.

The report can be viewed/downloaded.

---

89. DEMO REQUIREMENT

The prototype must have a polished demo path.

A new evaluator should be able to understand the product within 5 minutes.

Add a “Sample Case” seed.

Add clear navigation.

Do not require technical knowledge to operate the application.

---

90. FUTURE ROADMAP

After MVP:

Phase 2:

Excel import/export

OCR

document extraction

drawing interpretation

advanced rate management

approval workflow

email notifications

cloud deployment

S3 storage

PostgreSQL production configuration

role-specific dashboards

Phase 3:

AI-assisted measurement extraction

AI rate-item suggestion

AI document classification

AI anomaly detection

natural language search

analytics

multi-project reporting

---

91. FINAL ACCEPTANCE CRITERIA

The system is considered successful only when:

1. User can log in.
2. User can create a project.
3. User can create a valuation case.
4. User can enter owner/property information.
5. User can enter structure information.
6. User can create measurement formulas.
7. Quantity is calculated automatically.
8. Deductions are supported.
9. Construction items can be selected.
10. Rates come from database.
11. Abstract estimate is automatically generated.

12. Depreciation is calculated.
13. Salvage items can be selected.
14. Second valuation is calculated.
15. Final valuation is calculated.
16. Every important number is traceable.
17. Supporting documents can be uploaded.
18. Audit trail exists.
19. Calculation versions exist.
20. PDF is generated.
21. PDF contains the full valuation breakdown.
22. Seed case works end-to-end.
23. Tests pass.
24. Application runs with Docker.
25. README explains how to run the system.

---

92. FIRST ACTION REQUIRED FROM YOU

Do not immediately generate all files.

First produce:

1. Architecture plan
2. Database ER diagram description
3. Complete entity list
4. Calculation dependency diagram
5. API contract
6. Frontend route structure
7. Component hierarchy
8. Calculation engine design
9. PDF structure
10. Implementation roadmap

Then begin Phase 1 only.

After Phase 1 is completed and verified, continue to Phase 2.

At the end of every phase, provide:

Completed

What was implemented.

Files Changed

Exact files created/modified.

Database Changes

Migrations/entities changed.

APIs Added

Endpoints added.

Tests

Tests executed and results.

Issues

Any unresolved issue.

Next Phase

What will be implemented next.

Do not claim a phase is complete unless it has actually been implemented and tested.

---

93. MOST IMPORTANT RULE

The application is intended to automate a real engineering/valuation workflow.

Therefore:

The calculation engine must be deterministic, auditable, versioned and reproducible.

AI may assist users later, but AI must never silently modify or invent financial calculations.

Never replace source-derived business rules with assumptions.

Whenever a rule is ambiguous between the reference PDF and Excel workbook:

1. Preserve both source representations.
2. Mark the rule as requiring domain confirmation.
3. Make the rule configurable.
4. Do not silently choose a new formula.

The goal is not to recreate an Excel spreadsheet.

The goal is to build a professional digital estimation and valuation platform that reproduces validated calculations while making the entire workflow faster, safer, traceable and easier to operate.

BEGIN WITH PHASE 0.
Do not start implementation before presenting the architecture and calculation plan.

Sources